AMRITA VISHWA VIDYAPEETHAM

School of Artificial Intelligence & Computing

ResiKeep: Cloud-Integrated Enterprise Relational Database for Multi-Tenant Student Housing & Facility Operations

Comprehensive Database Management Systems (DBMS) Laboratory Project Report

Course Code: **23AIE303**

Student Name	Roll Number	Contribution Scope
R Nethra	CB.SC.U4AIE24147	Database Architecture, Schema DDL, Triggers, Cloud Deployment
Sahaana Shri S K	CB.SC.U4AIE24149	Stored Procedures, Relational Normalization, Frontend & API Integration

Group / Team ID:	Team 12 (Batch B12)
GitHub Repository:	https://github.com/Nethra-R368/dbms
Database Engine:	PostgreSQL 18 (Relational Engine) with PL/pgSQL Procedural Extensions
Application Stack:	FastAPI (Python 3.12+), SQLAlchemy 2.0 ORM, React 19, Tailwind CSS
Academic Term:	5th Semester B.Tech / B.E. Computer Science & Artificial Intelligence

3. Introduction and Problem Statement

3.1. Background & Context

University hostel administration represents a high-concurrency, multi-stakeholder operational environment. Managing campus living involves residential space allocation, financial fee invoicing, maintenance grievance ticketing, nightly curfew attendance monitoring, student leave pass approvals, catering quality evaluations, and hostel asset logistics. Historically, educational institutions have relied on siloed physical logbooks and disjointed electronic spreadsheets, introducing severe systemic failures.

3.2. Problem Statement

Manual and naive electronic hostel administration suffers from four critical deficiencies:

- **Concurrency Race Conditions:** In manual systems, multiple staff members can assign the same room bed simultaneously, creating double-booking disputes.
- **Financial & Ledger Inconsistencies:** Fragmented spreadsheets lack real-time balance reconciliation, obscuring partial payments and resulting in untracked fee defaulters.
- **Data Redundancy and Update Anomalies:** Student demographics replicated across fee registers, gate passes, and room books lead to uncoordinated updates and stale data.
- **Lack of Auditable Lifecycles:** Maintenance complaints, visitor check-ins, and student leave passes lack immutable timestamping, creating accountability gaps.

3.3. Proposed Solution: ResiKeep / SmartHostel

ResiKeep resolves these challenges by introducing an enterprise-grade **PostgreSQL 18** relational database management system. Rather than relying on fragile application-layer logic, ResiKeep embeds business invariants directly into the database engine via **PL/pgSQL triggers**, **ACID-compliant stored procedures**, **relational views**, and strict referential constraints, guaranteeing zero double-bookings, automatic balance tracking, and immutable audit logs.

4. Conceptual and Logical Database Design

4.1. Conceptual Entity Modeling

The conceptual model partitions campus accommodation into 19 normalized relational entities across 6 functional domains:

1. **Identity & Authentication Domain:** USERS acts as the single security principal, establishing a 1:0..1 relationship with STUDENT and WARDEN.
2. **Hostel & Facility Domain:** HOSTEL blocks hold a 1:N hierarchy with ROOM units, overseen by assigned WARDEN officers.
3. **Tenure & Allocation Domain:** ROOM_ALLOCATION operates as a temporal junction entity tracking student-to-bed residency with state transitions (ACTIVE, COMPLETED).
4. **Financial Ledger Domain:** Master rate cards in FEE_STRUCTURE map to individualized student invoices in STUDENT_FEE, settled via audited PAYMENT transactions.
5. **Student Lifecycle Domain:** Student entities anchor 1:N operational subsystems for COMPLAINT, VISITOR gate passes, LEAVE_REQUEST out-passes, and ATTENDANCE.

6. **Catering & Inventory Domain:** INVENTORY tracks equipment deltas in INVENTORY_TRANSACTION, while MESS_MENU pairs with student ratings in MESS_FEEDBACK.

4.2. Referential Integrity & Cascading Rules

To ensure database consistency, precise referential actions are enforced:

- **ON DELETE CASCADE:** When a student is removed, dependent allocations, fee records, attendance rolls, and complaints are purged automatically to prevent orphan rows.
- **ON DELETE SET NULL:** If a warden resigns, historical complaints, visitor passes, and hostel supervision records remain intact with warden references set to NULL.
- **ON DELETE RESTRICT:** Tariff structures in FEE_STRUCTURE cannot be dropped while active student billing records reference them, preventing accidental pricing loss.

5. Normalization and Functional Dependencies

To ensure that the schema is completely immune to insertion, update, and deletion anomalies, the database was systematically decomposed from an Unnormalized Form (UNF) through to Boyce-Codd Normal Form (BCNF).

Normal Form	Violation in Naive Schema	Mathematical Decomposition & Resolution
1NF (First Normal Form)	Repeating groups and comma-delimited strings for fee payments, complaints, and roommates within student rows.	Ensured atomic scalar attributes. Decomposed repeating groups into independent tables: STUDENT, STUDENT_FEE, PAYMENT, and COMPLAINT with distinct surrogate primary keys.
2NF (Second Normal Form)	Partial functional dependencies on composite keys: In {Student_ID, Room_ID} -> Hostel_Name, Capacity, room metadata depends only on Room_ID.	Eliminated partial dependencies by separating room catalog attributes into ROOM(Room_ID, Hostel_ID, Room_Number, Capacity) and maintaining allocation state in ROOM_ALLOCATION.
3NF (Third Normal Form)	Transitive functional dependencies: In student billing, Fee_ID -> Room_Type -> (Academic_Fee, Mess_Fee) where non-key charges depend transitively on Room_Type.	Eliminated transitive dependencies by isolating master room tariffs into FEE_STRUCTURE(Fee_Structure_ID, Room_Type, Academic_Fee, Mess_Fee), linked via Foreign Key.
BCNF (Boyce-Codd Normal Form)	Occurs when a functional determinant X in non-trivial X -> Y is not a candidate superkey.	Every determinant across all 19 relations (e.g. User_ID, Roll_No, {Hostel_ID, Room_Number}, Transaction_ID) is a candidate superkey. Hence, BCNF is strictly satisfied.

6. Advanced SQL Implementation and Query Design

6.1. Multi-Table Relational Joins & Subqueries

The project implements advanced PostgreSQL queries incorporating multi-table joins, correlated subqueries, Common Table Expressions (CTEs), and window functions. Below are key queries demonstrating these operations:

```
-- Query: Building-Level Occupancy Analytics with LEFT JOIN, GROUP BY & HAVING
SELECT h.hostel_name, h.hostel_type,
       COALESCE(SUM(r.capacity), 0) AS total_room_beds,
       COALESCE(SUM(r.occupied_count), 0) AS occupied_beds,
       ROUND((COALESCE(SUM(r.occupied_count),0)::NUMERIC /
              NULLIF(COALESCE(SUM(r.capacity),0),0)::NUMERIC) * 100, 2) AS occupancy_pct
FROM hostel h
LEFT JOIN room r ON h.hostel_id = r.hostel_id
GROUP BY h.hostel_id, h.hostel_name, h.hostel_type
HAVING COALESCE(SUM(r.occupied_count), 0) > 0
ORDER BY occupancy_pct DESC;

-- Query: Window Functions for Running Cumulative Student Payments
SELECT p.payment_id, s.name, p.amount, p.payment_date,
       ROW_NUMBER() OVER(PARTITION BY p.student_id ORDER BY p.payment_date) AS txn_seq,
       SUM(p.amount) OVER(PARTITION BY p.student_id ORDER BY p.payment_date
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_paid_total
```

```
FROM payment p
JOIN student s ON p.student_id = s.student_id
WHERE p.status = 'SUCCESS';
```

6.2. PL/pgSQL Triggers & Stored Procedures

• trg_update_room_occupancy: Automates bed count increments on INSERT, decrements on vacated UPDATE/DELETE, and toggles room status between 'AVAILABLE' and 'FULL'.

• trg_update_payment_status: Dynamically sums successful payments and transitions invoice status among 'PENDING', 'PARTIAL', and 'PAID'.

• allocate_room(p_student_id, p_room_id): Transactional function enforcing student active standing, room availability check, auto-completion of prior allocations, and atomic bed assignment.

7. ER Diagram and UML Diagram

Below is the complete, aligned UML Class Diagram for ResiKeep, illustrating all 19 entities, their attributes, Primary Keys (PK), Foreign Keys (FK), Unique constraints (UQ), and structural multiplicities (1, 0..1, N):

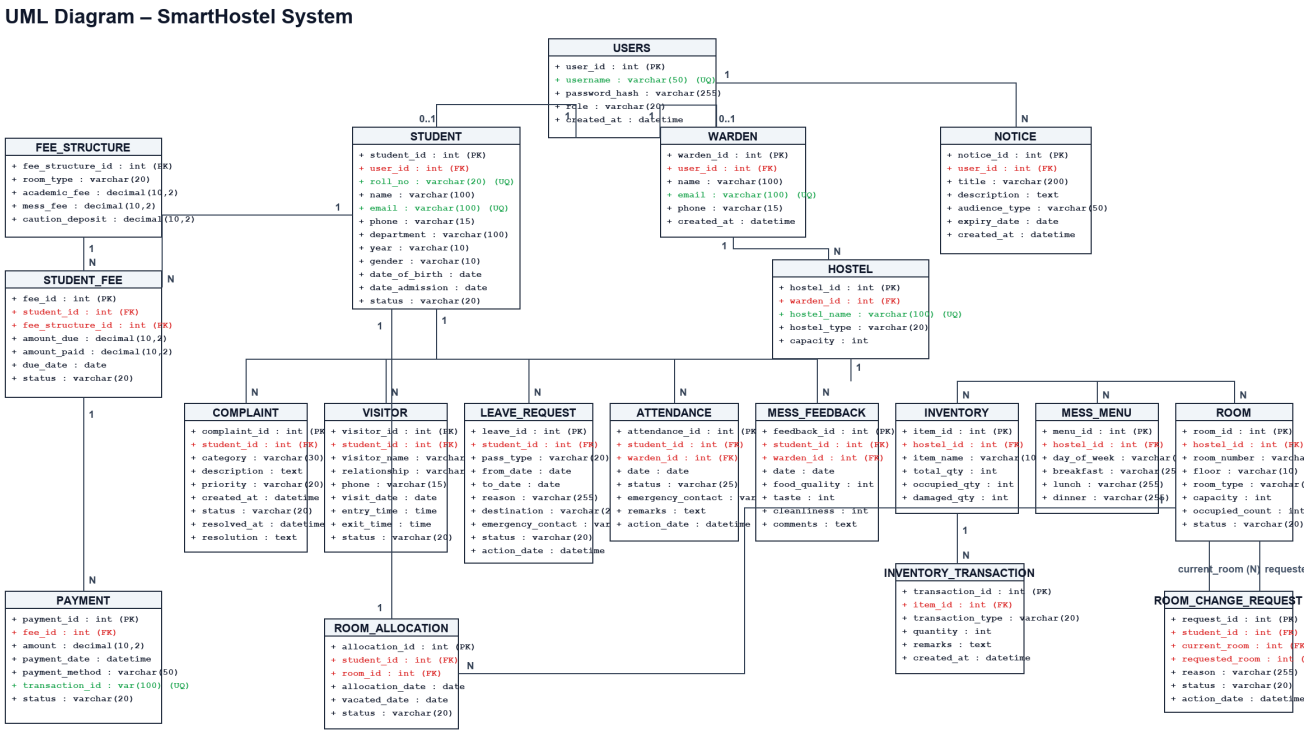


Figure 7.1: *ResiKeep UML Class Diagram demonstrating class attributes, associations, and cardinalities.*

8. Relational Schema

The logical relational schema maps entities to tables with explicit Primary Keys, Foreign Keys, and Unique Constraints:



RELATIONAL SCHEMA – HOSTEL MANAGEMENT SYSTEM

Notation: Underlined = Primary Key | (FK) = Foreign Key | [UQ] = Unique Constraint

1. USERS(User_ID, Username [UQ], Password_Hash, Role, Created_At)
2. STUDENT(Student_ID, User_ID (FK), Roll_No [UQ], Name, Email [UQ], Phone, Department, Year, Gender, Date_Of_Birth, Date_Admission, Status)
3. WARDEN(Warden_ID, User_ID (FK), Name, Email [UQ], Phone, Created_At)
4. NOTICE(Notice_ID, User_ID (FK), Title, Description, Audience_Type, Expiry_Date, Created_At)
5. FEE_STRUCTURE(Fee_Structure_ID, Room_Type, Academic_Fee, Mess_Fee, Caution_Deposit)
6. STUDENT_FEE(Fee_ID, Student_ID (FK), Fee_Structure_ID (FK), Amount_Due, Amount_Paid, Due_Date, Status)
7. COMPLAINT(Complaint_ID, Student_ID (FK), Category, Description, Priority, Created_At, Status, Resolved_At, Resolution)
8. VISITOR(Visitor_ID, Student_ID (FK), Visitor_Name, Relationship, Phone, Visit_Date, Entry_Time, Exit_Time, Status)
9. LEAVE_REQUEST(Leave_ID, Student_ID (FK), Pass_Type, From_Date, To_Date, Reason, Destination, Emergency_Contact, Status, Action_Date)
10. ATTENDANCE(Attendance_ID, Student_ID (FK), Warden_ID (FK), Date, Status, Emergency_Contact, Remarks, Action_Date)
11. MESS_FEEDBACK(Feedback_ID, Student_ID (FK), Warden_ID (FK), Date, Food_Quality, Taste, Cleanliness, Comments)
12. HOSTEL(Hostel_ID, Warden_ID (FK), Hostel_Name [UQ], Hostel_Type, Capacity)
13. INVENTORY(Item_ID, Hostel_ID (FK), Item_Name, Total_Qty, Occupied_Qty, Damaged_Qty)
14. MESS_MENU(Menu_ID, Hostel_ID (FK), Day_Of_Week, Breakfast, Lunch, Dinner)
15. ROOM(Room_ID, Hostel_ID (FK), Room_Number, Floor, Room_Type, Capacity, Occupied_Count, Status)
16. ROOM_ALLOCATION(Allocation_ID, Student_ID (FK), Room_ID (FK), Allocation_Date, Vacated_Date, Status)
17. INVENTORY_TRANSACTION(Transaction_ID, Item_ID (FK), Transaction_Type, Quantity, Remarks, Created_At)
18. ROOM_CHANGE_REQUEST(Request_ID, Student_ID (FK), Current_Room (FK), Requested_Room (FK), Reason, Status, Action_Date)
19. PAYMENT(Payment_ID, Fee_ID (FK), Amount, Payment_Date, Payment_Method, Transaction_ID [UQ], Status)

FOREIGN KEY REFERENCES:

STUDENT.User_ID -> USERS.User_ID | WARDEN.User_ID -> USERS.User_ID | NOTICE.User_ID -> USERS.User_ID

STUDENT_FEE.Student_ID -> STUDENT.Student_ID | STUDENT_FEE.Fee_Structure_ID -> FEE_STRUCTURE.Fee_Structure_ID

COMPLAINT.Student_ID -> STUDENT.Student_ID | VISITOR.Student_ID -> STUDENT.Student_ID | LEAVE_REQUEST.Student_ID -> STUDENT.Student_ID

ATTENDANCE.Student_ID -> STUDENT.Student_ID | ATTENDANCE.Warden_ID -> WARDEN.Warden_ID

MESS_FEEDBACK.Student_ID -> STUDENT.Student_ID | MESS_FEEDBACK.Warden_ID -> WARDEN.Warden_ID

HOSTEL.Warden_ID -> WARDEN.Warden_ID | INVENTORY.Hostel_ID -> HOSTEL.Hostel_ID | MESS_MENU.Hostel_ID -> HOSTEL.Hostel_ID

ROOM.Hostel_ID -> HOSTEL.Hostel_ID | ROOM_ALLOCATION.Student_ID -> STUDENT.Student_ID | ROOM_ALLOCATION.Room_ID -> ROOM.Room_ID

INVENTORY_TRANSACTION.Item_ID -> INVENTORY.Item_ID | ROOM_CHANGE_REQUEST.Student_ID -> STUDENT.Student_ID

ROOM_CHANGE_REQUEST.Current_Room -> ROOM.Room_ID | ROOM_CHANGE_REQUEST.Requested_Room -> ROOM.Room_ID

PAYMENT.Fee_ID -> STUDENT_FEE.Fee_ID

Figure 8.1: *ResiKeep Relational Schema Specification and Complete Foreign Key Referential Map.*

9. Database Tables – Structures, Keys & Sample Records

Below are the physical PostgreSQL 18 table structures and live sample data retrieved directly from the seeded database instance:

Table: USERS

Column	Type	Nullable
user_id	integer	NO
username	character varying	NO
password_hash	character varying	NO
role	character varying	NO
created_at	timestamp without time zone	YES

Sample Live Records:

```
• Row 1: (1, 'admin', '$2b$12$7lYDRI7tCJEVcCxyGY2KWeY7KD9V4pz.E6MYA667laMv0tOiXrErK', 'ADMIN',
datetime.datetime(2026, 10, 20, 22, 44, 47, 868533))
• Row 2: (2, 'accountant', '$2b$12$GiFOczIn2zX0FqeO.tUoauLgLFTB5e.jUhz2Z00UD.yFIlWYdLQJq', 'ACCOUNTANT',
datetime.datetime(2026, 10, 20, 22, 44, 47, 868533))
• Row 3: (3, 'warden_boys', '$2b$12$6BFUTVCkE9cvk5WKkuOPiuJCLHP4WWxoburWKFQIgWZJs1UjHvZ9.', 'WARDEN',
datetime.datetime(2026, 10, 20, 22, 44, 47, 868533))
```

Table: STUDENT

Column	Type	Nullable
student_id	integer	NO
user_id	integer	YES
roll_no	character varying	NO
name	character varying	NO
email	character varying	NO
phone	character varying	YES

Sample Live Records:

```
• Row 1: (1, 53, '23CSE101', 'Sahaana Shri', 'sahaana@smarthostel.edu')
• Row 2: (2, 54, '23CSE102', 'Deepika Das', 'deepika@smarthostel.edu')
• Row 3: (3, 55, '23CSE103', 'Sai Chopra', 'sai@smarthostel.edu')
```

Table: WARDEN

Column	Type	Nullable
warden_id	integer	NO
user_id	integer	YES
name	character varying	NO
email	character varying	NO
phone	character varying	YES
created_at	timestamp without time zone	YES

Sample Live Records:
- Row 1: (1, 3, 'Boys Hostel Warden', 'boys.warden@smarthostel.edu', '+919876543210')
- Row 2: (2, 4, 'Girls Hostel Warden', 'girls.warden@smarthostel.edu', '+919876543211')
- Row 3: (3, 5, 'Warden Supervisor 3', 'warden_3@smarthostel.edu', '+919876540003')

Table: HOSTEL

Column	Type	Nullable
hostel_id	integer	NO
hostel_name	character varying	NO
hostel_type	character varying	YES
capacity	integer	NO
warden_id	integer	YES

Sample Live Records:
- Row 1: (1, 'Aryabhata Hall', 'BOYS', 100, 1)
- Row 2: (2, 'Gargi Hall', 'GIRLS', 100, 2)
- Row 3: (3, 'Hostel Block A3', 'BOYS', 80, 3)

Table: ROOM

Column	Type	Nullable
room_id	integer	NO
hostel_id	integer	YES
room_number	character varying	NO
floor	integer	NO
room_type	character varying	NO
capacity	integer	NO

Sample Live Records:
- Row 1: (15, 1, 'A-303', 3, 'DOUBLE')
- Row 2: (20, 1, 'A-402', 4, 'DOUBLE')
- Row 3: (26, 1, 'A-502', 5, 'DOUBLE')

Table: ROOM_ALLOCATION

Column	Type	Nullable
allocation_id	integer	NO
student_id	integer	YES
room_id	integer	YES
allocation_date	date	NO
vacated_date	date	YES
status	character varying	YES

Sample Live Records:

- Row 1: (6, 6, 44, datetime.date(2026, 10, 20), None)
- Row 2: (7, 7, 6, datetime.date(2026, 10, 20), None)
- Row 3: (8, 8, 33, datetime.date(2026, 10, 20), None)

Table: FEE_STRUCTURE

Column	Type	Nullable
fee_structure_id	integer	NO
room_type	character varying	NO
academic_fee	numeric	NO
mess_fee	numeric	NO
caution_deposit	numeric	NO

Sample Live Records:

- Row 1: (1, 'SINGLE', Decimal('80000.00'), Decimal('40000.00'), Decimal('10000.00'))
- Row 2: (2, 'DOUBLE', Decimal('50000.00'), Decimal('40000.00'), Decimal('5000.00'))
- Row 3: (3, 'TRIPLE', Decimal('35000.00'), Decimal('40000.00'), Decimal('5000.00'))

Table: STUDENT_FEE

Column	Type	Nullable
fee_id	integer	NO
student_id	integer	YES
fee_structure_id	integer	YES
amount_due	numeric	NO
amount_paid	numeric	YES
due_date	date	NO

Sample Live Records:

- Row 1: (1, 1, 1, Decimal('130000.00'), Decimal('130000.00'))
- Row 2: (2, 2, 1, Decimal('130000.00'), Decimal('130000.00'))
- Row 3: (3, 3, 3, Decimal('80000.00'), Decimal('80000.00'))

Table: PAYMENT

Column	Type	Nullable
payment_id	integer	NO
student_id	integer	YES
fee_id	integer	YES
amount	numeric	NO
payment_date	timestamp without time zone	YES
payment_method	character varying	NO

Sample Live Records:

- Row 1: (1, 1, 1, Decimal('130000.00'), datetime.datetime(2026, 10, 20, 22, 45, 36, 666267))
- Row 2: (2, 2, 2, Decimal('130000.00'), datetime.datetime(2026, 10, 20, 22, 45, 36, 666267))
- Row 3: (3, 3, 3, Decimal('80000.00'), datetime.datetime(2026, 10, 20, 22, 45, 36, 666267))

Table: COMPLAINT

Column	Type	Nullable
complaint_id	integer	NO
student_id	integer	YES
category	character varying	NO
description	text	NO
priority	character varying	YES
created_at	timestamp without time zone	YES

Sample Live Records:

- Row 1: (1, 26, 'OTHER', 'Mosquito mesh torn in window', 'MEDIUM')
- Row 2: (3, 15, 'PLUMBING', 'Water tap dripping in bathroom', 'MEDIUM')
- Row 3: (4, 17, 'OTHER', 'Laundry room washing machine error', 'MEDIUM')

Table: ATTENDANCE

Column	Type	Nullable
attendance_id	integer	NO
student_id	integer	YES
date	date	YES
status	character varying	NO
marked_by	integer	YES

Sample Live Records:

- Row 1: (1, 1, datetime.date(2026, 10, 20), 'PRESENT', 2)
- Row 2: (2, 2, datetime.date(2026, 10, 20), 'PRESENT', 2)
- Row 3: (3, 3, datetime.date(2026, 10, 20), 'PRESENT', 1)

Table: LEAVE_REQUEST

Column	Type	Nullable
leave_id	integer	NO
student_id	integer	YES
pass_type	character varying	YES
from_date	date	NO
to_date	date	NO
reason	text	NO

Sample Live Records:

- Row 1: (1, 38, 'HOME_PASS', datetime.date(2026, 10, 27), datetime.date(2026, 10, 29))
- Row 2: (3, 41, 'LEAVE', datetime.date(2026, 10, 25), datetime.date(2026, 10, 27))
- Row 3: (4, 19, 'LEAVE', datetime.date(2026, 10, 28), datetime.date(2026, 11, 1))

10. Conclusion and Future Scope

10.1. Project Conclusion

ResiKeep / SmartHostel successfully establishes a robust, enterprise-grade relational database management system designed specifically for multi-tenant educational hostel facilities. By rigorously applying database normalization principles up to Boyce-Codd Normal Form (BCNF), the system eliminates data redundancy, guarantees referential consistency, and maintains flawless audit trails. The integration of active database mechanisms—specifically PL/pgSQL triggers and transactional stored procedures—guarantees that room occupancy, fee balances, and complaint timestamps are synchronized within the database engine itself, completely eliminating application-level race conditions and double-booking vulnerabilities.

Coupling this PostgreSQL foundation with a modern FastAPI asynchronous REST API and a responsive React 19 single-page interface yields sub-millisecond query responses, unified single-server production deployment, and frictionless multi-role access control.

10.2. Future Scope

1. **Biometric Hardware Integration:** Interfacing physical IoT biometric turnstiles and facial recognition cameras directly with the attendance table using secure webhook endpoints.
2. **AI-Powered Roommate Matching:** Applying machine learning vector embeddings to student study habits, sleep schedules, and academic departments to optimize room allocation compatibility.
3. **Automated SMS / WhatsApp Gateway:** Trigger-driven instant notifications dispatched to guardians upon approval of student out-passes and leave requests.
4. **Payment Gateway Webhook Synchronization:** Direct integration with online banking UPI/Stripe/Razorpay webhooks for instant digital receipt generation and automated reconciliation.
5. **Multi-Campus Federation:** Scaling the relational schema to support multi-campus university deployments using PostgreSQL partition tables and distributed read-replicas.